HDOC Day-64

Finding the Duplicate Element Using Pointers and XOR in C

Aim

To write a C program using a variable-length integer array, pointers, a user-defined function, and bitwise XOR to identify the only repeated element.

Input Assumption

The array contains **size** elements: the integers 1 through size - 1, with exactly one integer appearing twice. All other elements appear once. The XOR approach relies on this assumption.

Algorithm

1. Read and validate the array size (size must be greater than 1).
2. Declare nums[size] and read its elements.
3. Pass the array address and size to duplicate_element().
4. Initialize xorResult to zero.
5. XOR all elements of the array using pointer arithmetic.
6. XOR integers 1 through size - 1 with xorResult.
7. All numbers appearing twice in the combined XOR cancel; the remaining value is the duplicate.
8. Display the duplicate element and stop.

C Program

```
#include <stdio.h>

void duplicate_element(int *nums, int size);

int main()
{
    int size, i;

    printf("Enter the size of array: ");
    scanf("%d", &size);

    if (size <= 1)
    {
        printf("Invalid array size!\n");
        return 0;
    }

    int nums[size];
    printf("Enter %d array elements: ", size);

    for (i = 0; i < size; i++)
    {
        scanf("%d", &nums[i]);
    }

    duplicate_element(nums, size);
    return 0;
}

/* Function to find duplicate using XOR */
void duplicate_element(int *nums, int size)
{
    int i;
    int xorResult = 0;

    /* XOR all array elements */
    for (i = 0; i < size; i++)
    {
        xorResult ^= *(nums + i);
    }

    /* XOR expected integers from 1 to size - 1 */
    for (i = 1; i < size; i++)
    {
        xorResult ^= i;
    }

    printf("\nDuplicate Element: %d\n", xorResult);
}
```

Explanation

The function **duplicate_element(int *nums, int size)** receives the array address. The expression ***(nums + i)** is equivalent to nums[i]. XOR has the properties a ^ a = 0 and a ^ 0 = a. Thus, combining the array elements with the expected values 1 through size - 1 cancels every nonduplicate element.

Example

```
Input array: 1 3 4 2 2
Expected:    1 2 3 4
Duplicate Element: 2
```

Complexity

Time complexity: **O(n)**. Auxiliary space complexity: **O(1)**. The function makes two linear passes and does not require sorting or an extra frequency array.

Test Case Table

Case	Input array	Size	Expected
1	1 3 4 2 2	5	2
2	3 1 3 4 2	5	3
3	1 1	2	1
4	1 2 3 4 4	5	4
5	2 1 2 3 4	5	2
6	1 2 3 3	4	3
7	4 2 1 3 4	5	4
8	2 2 1	3	2
9	1 2 3 4 5 5	6	5
10	size = 1	1	Invalid size

Sample Outputs

Test Case 1: Duplicate in the array

```
Enter the size of array: 5
Enter 5 array elements: 1 3 4 2 2

Duplicate Element: 2
```

Test Case 2: Another duplicate

```
Enter the size of array: 5
Enter 5 array elements: 3 1 3 4 2

Duplicate Element: 3
```

Test Case 3: Smallest valid array

```
Enter the size of array: 2
Enter 2 array elements: 1 1

Duplicate Element: 1
```

Test Case 4: Duplicate at the end

```
Enter the size of array: 6
Enter 6 array elements: 1 2 3 4 5 5

Duplicate Element: 5
```

Test Case 5: Invalid size

```
Enter the size of array: 1
```

```
Invalid array size!
```

Result

The C program uses a variable-length array, pointers, a user-defined function, and XOR operations to identify the repeated element. Under the specified range and frequency assumptions, it runs in O(n) time and O(1) auxiliary space.